

Pixel Art on an 8×8 LED Matrix

IoT Foundations Lab — Phases 1 & 2

Created by **Danish Puri** · Liberty College · IoT Program

The Problem

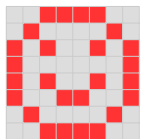
How does a computer display a picture? Every image on every screen — from your phone to a billboard — is just a grid of tiny colored lights called **pixels**. In this lab, you will build a miniature version: a physical 8×8 grid of 64 LEDs wired to an Arduino. You will code it to display *static pixel art* and *animations*, giving you hands-on insight into how graphics, binary data, and hardware work together.

What You Are Building

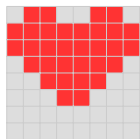
A **Pixel Art Gallery** that cycles through images and animations on a 64-LED display driven by an Arduino UNO. Each image is stored as eight rows of binary data — 1 = LED ON, 0 = LED OFF — and the Arduino refreshes them fast enough that your eye sees a complete picture.

Pixel Art Preview

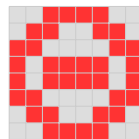
Below are four designs you will program. Each tiny square represents one LED.



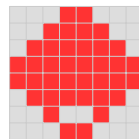
Smiley



Heart



Skull



Arrow Up

Tools & Requirements

Hardware

- Arduino UNO × 1
- 8×8 LED Matrix with MAX7219 driver module × 1 (choose red, green, blue, or yellow — each module is one color)
- Half-size breadboard × 1
- Jumper wires × 5 (male-to-female)
- USB-A to USB-B cable (to program the Arduino)

Note

The MAX7219 module has the multiplexing driver *built in* — it controls all 64 LEDs using only 3 Arduino pins, so you do **not** need to wire each LED individually.

Software

- Arduino IDE (free at arduino.cc)
- Library: **LedControl** — install via *Sketch* → *Include Library* → *Manage Libraries*

Phase 1 — Wire the Circuit

How the MAX7219 Works

The MAX7219 is a *serial-to-parallel driver*: you send it data one bit at a time over three wires (SPI protocol) and it handles turning on the correct 64 LEDs. Without it, you would need a separate pin for every LED — impossible on a UNO.

Wiring Table

MAX7219 Pin	Arduino Pin	Purpose
VCC	5V	Power
GND	GND	Ground
DIN	11	Data in (MOSI)
CS	10	Chip select
CLK	13	Clock (SCK)

Breadboard Schematic (Tinkercad)

Tip

Tinkercad step-by-step:

1. Go to tinkercad.com and create a free account.
2. Click **Create** → **Circuit**.
3. Search *Arduino UNO* and drag it to the canvas.
4. Search *8x8 LED Matrix MAX7219* and drag it beside the breadboard.
5. Connect wires following the table above (use colors: red=VCC, black=GND, blue=DIN, yellow=CS, green=CLK).
6. Click **Start Simulation** to test before building real hardware.

The physical wiring mirrors the schematic exactly. Seat the MAX7219 module on the breadboard, then use five jumper wires to connect it to the Arduino per the table above. Double-check VCC and GND before powering on.

Phase 2 — Code the Pixel Art Gallery

How Images Are Stored

Each image is an array of 8 bytes. Each byte = one row of 8 LEDs. A 1 bit lights the LED; a 0 bit turns it off. This is literally how every computer screen works — just at a much smaller scale.

Full Arduino Code

Copy this template into the Arduino IDE, then upload it to your board.

Listing 1: Pixel Art Gallery — starter template

```
1 #include
2
3 // LedControl(DIN, CLK, CS, numDevices)
```

```

4
5
6 //      Pixel art patterns (8 rows x 8 cols)
7
8
9
10
11
12 //      Animation frames: blinking eye
13
14
15 //      Helper: display one pattern
16
17 void
18   for int
19
20
21
22 //      Helper: clear display
23
24 void
25   for int
26
27
28 //      Bouncing ball state
29
30 int
31 int
32 void      int
33   for int
34
35           true
36
37
38   if
39   if
40
41
42
43
44 void
45           false      // wake up MAX7219
46                       // brightness 0-15
47
48
49
50 void
51   // -- Static gallery --
52
53
54
55
56
57   // -- Blinking eye animation (10 blinks) --
58   for int
59
60
61
62
63   // -- Bouncing ball (60 frames) --
64
65

```

convert to hex bytes, add to the gallery.

- **Sensor-reactive:** connect a microphone; bar-graph the volume level live.

Tip

Hex cheat: convert a row of 8 bits to hex by grouping into two nibbles.

Example: 0b10100101 → 1010 = A, 0101 = 5 → 0xA5

Deliverables

1. Tinkercad screenshot of your wired circuit (Phase 1).
2. Working Arduino sketch with at least **three static images** and **one animation** of your own design (Phase 2).
3. One paragraph: *How does binary data become a visible image?*

Ideas to Expand

- **Raindrop:** move lit pixels downward row by row, reset to top — simulates falling rain.
- **Wave:** use a sine lookup table to ripple a horizontal line up and down.
- **Snake game:** add a button; the lit pixel "grows" a tail and you steer it.
- **Your own icon:** design anything on 8×8 graph paper,